

Telangana District Map Coloring Using AC-3

1. Introduction

Map coloring is a classical problem in **Artificial Intelligence** and **Constraint Satisfaction Problems (CSPs)**.

The basic idea is to color different regions of a map so that two regions sharing a common boundary do not have the same color.

In this project, the problem is applied to the **33 districts of Telangana**.

Instead of manually assigning colors, the problem is solved using an AI-based approach involving:

- Constraint Satisfaction
- AC-3 Arc Consistency
- Backtracking Search
- Graph representation

The final result provides a valid color assignment where neighbouring districts have different colors.

2. Problem Statement

The objective is to implement the **Map Coloring Problem for all 33 Telangana districts using the AC-3 algorithm**.

Each district must receive one color from a predefined set of colors.

The main condition is:

If District A and District B are neighbours,

$\text{Color}(A) \neq \text{Color}(B)$

The algorithm should find a valid assignment satisfying all such constraints.

3. Objectives

The main objectives of the experiment are:

1. Understand the Map Coloring Problem.
 2. Formulate the problem as a CSP.
 3. Represent Telangana districts as variables.
 4. Define possible colors as domains.
 5. Define neighbouring districts as constraints.
 6. Implement the AC-3 algorithm.
 7. Apply backtracking for complete assignment.
 8. Verify the final coloring.
 9. Visualize the district relationships.
 10. Understand practical applications of CSPs.
-

4. Constraint Satisfaction Problem

A Constraint Satisfaction Problem consists of three main components:

`CSP = Variables + Domains + Constraints`

For this project:

Variables

The variables are the 33 Telangana districts.

Domains

Each district initially has:

`{Red, Green, Blue, Yellow}`

Constraints

Adjacent districts cannot have the same color.

Therefore:

`Color(X) ≠ Color(Y)`

whenever `x` and `y` are neighbouring districts.

5. Telangana District Variables

The project considers all 33 districts:

Adilabad

Because Hyderabad and Rangareddy are adjacent, they cannot finally have the same color.

7. Adjacency Constraints

The neighbouring districts are represented using an adjacency dictionary.

For example:

```
"Hyderabad": {  
    "Medchal-Malkajgiri",  
    "Rangareddy"  
}
```

This means Hyderabad has constraints with:

```
Medchal-Malkajgiri  
Rangareddy
```

The corresponding constraint is:

```
Hyderabad ≠ Medchal-Malkajgiri  
Hyderabad ≠ Rangareddy
```

The same approach is used for all districts.

8. Graph Representation

The district problem can also be represented as a graph.

District = Node

Neighbouring relationship = Edge

For example:

```
Hyderabad ----- Rangareddy  
    |  
    |  
    |  
Medchal-Malkajgiri
```

The graph therefore represents the constraints of the CSP.

9. Why Four Colors?

Map-coloring problems are commonly demonstrated using four colors.

Using four colors gives every district enough possible choices while keeping the problem simple.

The domain is:

`D = {Red, Green, Blue, Yellow}`

The algorithm can then eliminate colors whenever they violate constraints.

10. AC-3 Algorithm

AC-3 stands for:

Arc Consistency Algorithm 3

It is used to reduce the possible values in CSP domains.

The main idea is:

Every value in the domain of one variable should have at least one compatible value in the domain of its neighbouring variable.

Consider:

`District A → District B`

Suppose:

`A = {Red}`

`B = {Red}`

Because adjacent districts cannot have the same color, `Red` is not a valid value for A.

Therefore:

`A = {}`

This indicates a conflict.

11. AC-3 Processing

AC-3 maintains a queue of arcs.

An arc is represented as:

(X_i, X_j)

The algorithm repeatedly performs:

1. Remove an arc from the queue.
 2. Check whether X_i needs revision.
 3. Remove unsupported values.
 4. If X_i changes:
 Add affected neighbouring arcs.
 5. Continue until the queue is empty.
-

12. The `revise()` Function

The `revise()` function is responsible for removing unsupported values.

Conceptually:

```
for x in domain[Xi]:  
    if no compatible y exists in domain[Xj]:  
        remove x
```

For map coloring, compatibility means:

$x \neq y$

because neighbouring districts must have different colors.

13. The `ac3()` Function

The AC-3 function first creates a queue containing all directed district relationships.

Example:

```
(Hyderabad, Rangareddy)  
(Rangareddy, Hyderabad)
```

The algorithm then processes each relationship.

If a domain changes, neighbouring relationships are added back to the queue.

This continues until the CSP becomes arc-consistent.

14. Why Backtracking is Required

An important point is that **AC-3 alone is a constraint propagation algorithm.**

It reduces domains, but it does not necessarily select one final color for every district.

For example, after AC-3:

Hyderabad:
{Red, Blue}

Rangareddy:
{Green, Yellow}

The domains are consistent, but each district still has two possible colors.

Therefore, the project combines AC-3 with **backtracking search**.

The process becomes:

```
AC-3
↓
Reduce domains
↓
Choose a district
↓
Try a color
↓
Run AC-3 again
↓
Continue
```

15. Backtracking Search

Backtracking is a systematic trial-and-error approach.

The algorithm:

1. Selects an unassigned district.
2. Chooses a possible color.
3. Checks whether the color violates an existing constraint.
4. If valid, assigns the color.
5. Runs AC-3.
6. Continues to the next district.
7. If a conflict occurs, it goes back.
8. Tries another color.

This continues until all districts are assigned.

16. Minimum Remaining Values

The implementation uses the **Minimum Remaining Values (MRV)** idea.

Instead of randomly selecting a district, the algorithm chooses the unassigned district having the smallest remaining domain.

For example:

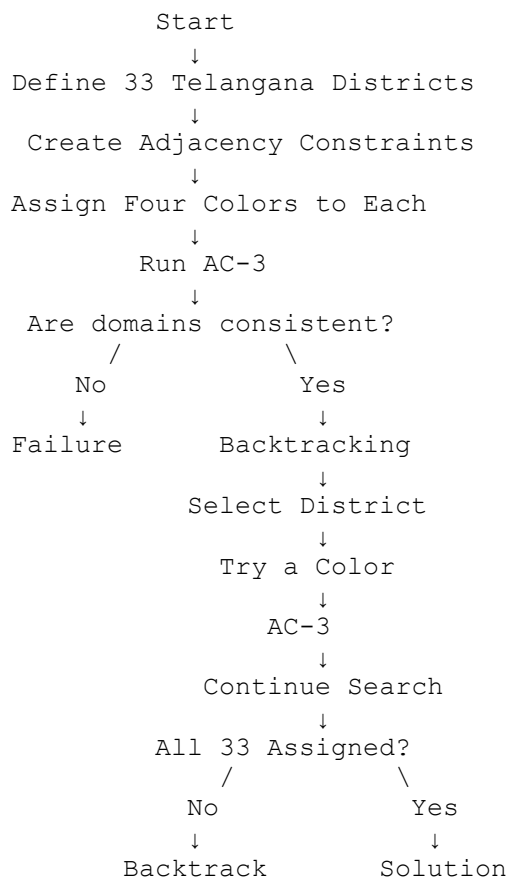
District A $\rightarrow$ {Red, Blue}
District B $\rightarrow$ {Red, Green, Blue}
District C $\rightarrow$ {Green}

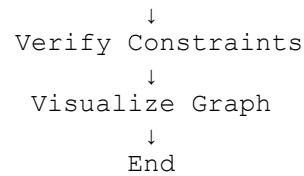
District C is selected first because it has only one remaining option.

This can make the search more efficient.

17. Complete Algorithm Flow

The complete project follows this flow:





18. Verification

After obtaining the solution, every adjacency relationship is checked.

For every pair:

District A ↔ District B

the program checks:

`solution[A] != solution[B]`

If all relationships satisfy this condition:

VALID MAP COLORING

is displayed.

19. Visualization

NetworkX is used to visualize the district constraint graph.

The visualization represents:

Node → Telangana district

Edge → Neighbouring relationship

Node color → Assigned district color

This makes the solution easier to understand visually.

The four colors are represented using:

Red
Green
Blue
Yellow

20. Expected Result

The final program should display something similar to:

```
🎉 SUCCESS!  
All 33 Telangana districts have been colored.
```

The verification stage should display:

```
✅ VALID MAP COLORING  
No adjacent districts have the same color.
```

A table is also generated showing:

District	Color
Adilabad	Red
Bhadradi Kothagudem	Green
Hanumakonda	Blue
Hyderabad	Red
...	

The exact color assigned to each district can vary because multiple valid colorings may exist.

21. Advantages of AC-3

AC-3 provides several benefits:

1. Reduces Search Space

Invalid colors are removed before the complete search.

2. Detects Conflicts Early

An empty domain indicates that the current assignment cannot work.

3. Improves Backtracking

Backtracking has fewer choices to explore after constraint propagation.

4. Easy to Understand

The algorithm directly represents the CSP constraints.

5. General Purpose

AC-3 can be applied to many other CSP problems.

22. Limitations

The approach also has some limitations:

1. The adjacency relationships must be accurate.
 2. AC-3 alone may not produce a complete assignment.
 3. Backtracking may still become expensive for larger CSPs.
 4. The graph visualization represents relationships rather than an exact geographic map.
 5. Changes in district boundaries would require updating the adjacency data.
-

23. Real-World Applications

The same idea can be applied to:

Map Coloring

Coloring administrative regions.

Timetable Scheduling

Assigning subjects to time slots without conflicts.

Examination Scheduling

Avoiding examination clashes.

Frequency Assignment

Assigning frequencies to nearby communication towers.

Route Planning

Managing conflicting routes or resources.

Classroom Allocation

Assigning classes to rooms and times.

Artificial Intelligence

Solving general constraint satisfaction problems.

24. Experimental Procedure

The experiment was performed in the following stages.

Stage 1 — Define Districts

All 33 Telangana districts were stored as CSP variables.

Stage 2 — Define Neighbours

Neighbouring district relationships were represented using an adjacency dictionary.

Stage 3 — Define Domains

Each district received the initial domain:

```
{Red, Green, Blue, Yellow}
```

Stage 4 — Apply AC-3

The algorithm removed colors that were inconsistent with neighbouring districts.

Stage 5 — Backtracking

When multiple colors remained, backtracking was used to find a complete assignment.

Stage 6 — Verification

Every neighbouring pair was checked.

Stage 7 — Visualization

NetworkX and Matplotlib were used to display the resulting colored graph.

25. Observation

The experiment demonstrates that the Telangana district map can be formulated effectively as a **Constraint Satisfaction Problem**.

AC-3 reduces the available color choices by enforcing consistency between neighbouring districts. When AC-3 cannot uniquely assign all districts, backtracking completes the assignment.

The final verification confirms that no two adjacent districts have the same color.

Therefore, the combination of **AC-3 + Backtracking** provides an effective AI-based solution for the Telangana district map-coloring problem.

26. Conclusion

The Telangana District Map Coloring problem was successfully implemented using the principles of **Constraint Satisfaction Problems**.

All 33 Telangana districts were represented as variables, and neighbouring relationships were represented as constraints. Four colors were provided as possible domain values.

The **AC-3 algorithm** was used to enforce arc consistency and reduce invalid color assignments. **Backtracking search** was then used to complete the coloring when multiple choices remained.

Finally, the solution was verified to ensure that adjacent districts did not receive the same color.

This experiment demonstrates a practical application of Artificial Intelligence and provides a clear understanding of:

```
CSP
↓
Domains
↓
Constraints
↓
AC-3
↓
Backtracking
↓
Valid Solution
```
